Министерство цифрового развития, связи и массовых коммуникаций Российской Федерации федеральное государственное бюджетное образовательное учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»

Кафедра вычислительных систем

ОТЧЕТ
по практической работе 5

по дисциплине **«Основы разработки на Java»**

Выполнил:
студент гр. ИС-542
«11» октября 2026 г. _____ /Горбачев Д.В./

Проверил:
Ассистент кафедры ВС
«11» октября 2026 г. _____ /Гузенков С.К./

Оценка «_____»

Новосибирск 2026

Ссылка на репозиторий: _____

Задание 1 List и порядок

Сохраните последовательность регистраций, вставьте и удалите элемент, найдите дубликаты. Сравните ArrayList и LinkedList для чтения по индексу и обхода; выбор подтвердите измерением без заявления о микробенчмарке.

В Task01Lists список ArrayList хранит регистрации в порядке добавления. Элемент вставляется по индексу 1, регистрация студента 2 на курс 101 удаляется по значению. Повтором считается одинаковая пара studentId/courseId; обнаружена регистрация (1, 101). Поиск повторов через contains в этом небольшом примере имеет сложность $O(n^2)$.

Оба списка заполнены одинаковыми 10 000 регистрациями. После предварительного запуска измерен полный проход через get(i) и через for-each. Заполнение и вывод не входят в замер; сумма ID во всех случаях равна 50 005 000. Результаты одного запуска JVM 26 приведены ниже.

Операция	ArrayList мс	LinkedList мс
Чтение всех элементов по индексу	0,368	43,372
Полный обход for-each	0,286	0,131

Для индексного чтения выбран ArrayList: get(i) работает за O(1), а у LinkedList - за O(n). Полный обход через индексы занимает соответственно O(n) и $O(n^2)$; for-each у обоих - O(n). Это учебное измерение, не микробенчмарк: время зависит от JVM и нагрузки, а один запуск не доказывает устойчивое преимущество при обходе.

Задание 2 Set и уникальность

Удалите дубликаты CourseCode, сохранив первый порядок появления. Выберите реализацию set и объясните влияние equals/hashCode

Использован LinkedHashSet<CourseCode>: из JAVA, SQL, JAVA, PYTHON, SQL получено JAVA, SQL, PYTHON в порядке первого появления. Обычный HashSet этот порядок не гарантирует. CourseCode - record, поэтому equals и hashCode учитывают value. Хеш определяет корзину поиска, а equals подтверждает равенство: коллизия сама по себе не делает элементы одинаковыми. Равные объекты должны иметь одинаковый хеш; поиск и добавление ожидаемо O(1) при хорошем распределении хешей.

Задание 3 Map и индекс

Постройте индекс CourseId -> Course, обработайте повторный ID без молчаливой перезаписи. Сравните get, containsKey, putIfAbsent и computeIfAbsent.

Индекс CourseId → Course построен на LinkedHashMap для воспроизводимого порядка вывода. get возвращает курс либо null, containsKey проверяет наличие ключа. putIfAbsent добавляет готовый объект, если значения нет; при повторном ID возвращает прежний курс, после чего addCourse выбрасывает IllegalArgumentException. Курс 101 остается Java, молчаливой замены нет.

computeIfAbsent вызывает функцию создания только при отсутствии значения: Python с ID 103 создается один раз. Оба метода считают значение null отсутствующим; наш индекс null-значений не содержит. CourseId как record обеспечивает согласованные equals/hashCode.

Задание 4 Очередь ожидания

Реализуйте FIFO waitlist через Queue: добавить, посмотреть следующего, извлечь, обработать пустую очередь. Объясните пары add/offer, remove/poll, element/peek.

Очередь Queue<Registration> реализована через ArrayDeque. Регистрации 1, 2, 3 извлекаются в том же порядке: FIFO. add и offer добавляют в конец; в заполненной ограниченной очереди add выбрасывает IllegalStateException, offer возвращает false. ArrayDeque расширяется автоматически.

remove и poll возвращают и удаляют первого; при пустоте remove выбрасывает NoSuchElementException, poll возвращает null. element и peek только смотрят первого: при пустоте соответственно исключение и null. Null-элементы запрещены; поэтому null от poll/peek означает пустую очередь. Операции на концах амортизированно O(1).

Задание 5 Безопасное удаление при обходе

Воспроизведите ConcurrentModificationException при удалении из list в enhanced for. Исправьте через iterator и через removeIf. Объясните, почему копирование всей коллекции не всегда лучший default.

Прямое удаление из ArrayList внутри enhanced for вызывает ConcurrentModificationException при продолжении обхода. Первая регистрация уже удалена: исключение не откатывает изменение. Это возможно в одном потоке; обнаружение такого изменения не гарантировано во всех случаях.

Исправления: iterator.remove удаляет последний элемент, полученный next, и согласует состояние итератора; removeIf удаляет все элементы по условию courseId == 101. В обоих случаях остаются регистрации (2, 102) и (4, 103). Копия списка требует O(n) дополнительной памяти и не устраняет стоимость повторных удалений из оригинала, поэтому для простого условия достаточно removeIf.

Задание 6 Первый обобщенный тип

Реализуйте Result<T> или Page<T> без raw types. Добавьте фабрики успеха/ошибки, инварианты и методы чтения. Покажите две конкретизации и compile-time защиту от смешения.

Result<T> создается только через success и failure. Инвариант: присутствует ровно одно - ненулевое значение или непустая причина ошибки. value нельзя читать у отказа, error - у успеха; нарушение вызывает IllegalStateException. Конкретизации Result<Course> и Result<String> сохраняют тип значения без raw types. Несовместимое присваивание ниже отвергается компилятором.

```
Result<Course> course = Result.success(
    new Course(101, "Java", 40));
Result<String> message = Result.success("Готово");
/* Ошибка компиляции: Result<String> wrong = course; */
```

Задание 7 Обобщенные методы

Напишите firstOrThrow(List<T>), swap(List<T>, int, int) и indexBy(List<T>, Function<T,K>). Последний метод должен явно обрабатывать конфликт ключей.

firstOrThrow(List<T>) возвращает первый элемент либо выбрасывает NoSuchElementException для пустого списка. swap проверяет оба индекса до изменения и меняет значения через временную переменную; список должен поддерживать set.

indexBy(List<T>, Function<T,K>) вычисляет ключ для каждого элемента и строит LinkedHashMap с сохранением порядка. Null-элементы и ключи запрещены. Повторный ключ явно вызывает IllegalArgumentException; для двух регистраций студента 1 получена ошибка «Повторный ключ: 1». Это индекс одной регистрации на ключ, а не группировка нескольких регистраций.

Задание 8 Ограничение типа

Реализуйте max для элементов Comparable. Корректно сформулируйте bound с учётом подтипов. Проверьте на Integer, String и собственном value object.

Метод max имеет ограничение T extends Comparable<? super T>. Оно позволяет сравнение, реализованное для самого типа или его предка: Child подходит, если Parent реализует Comparable<Parent>. compareTo возвращает отрицательное число, ноль или положительное число для меньшего, равного или большего значения.

Проверки дали 12 для Integer, SQL для String и 305 для CourseId. Строки сравниваются лексикографически, ID - через Long.compare без переполнения вычитания. Пустой список и null-элементы запрещены. Список не изменяется; обход O(n), дополнительная память O(1).

Задание 9 PECS на практике

Создайте методы копирования из producer-списка в consumer-список. Объясните, почему нельзя добавлять конкретный объект в List<? extends Course> и что разрешено читать из List<? super Course>.

PECS означает Producer Extends, Consumer Super. Метод copy принимает List<? extends T> как источник и List<? super T> как получатель. Показано копирование List<OnlineCourse> в List<Course>, затем в List<Object>. Добавляются ссылки на те же объекты; источник не очищается.

Из List<? extends Course> безопасно читать Course, но нельзя добавлять конкретный объект: реальный тип может оказаться неизвестным более узким наследником. В List<? super Course> можно добавлять Course и наследников; при чтении гарантирован только Object, поскольку реальный список может содержать и другие объекты. Получатель должен поддерживать add.

Задание 10 Контракт ключа

Проверьте сущности Course в HashSet и как ключи HashMap. Измените поле, не участвующее в равенстве, затем намеренно включите его в hash и воспроизведите проблему.

Course.equals и hashCode зависят только от ID. После completeHours(6) хеш остается прежним, HashSet находит равный по ID курс, HashMap возвращает значение по равному ключу. Изменяемый прогресс не участвует в равенстве.

В демонстрационном BrokenHashCourse прогресс намеренно добавлен в хеш. После его изменения contains возвращает false, get - null, хотя размер каждой коллекции остается 1. Запись осталась в корзине прежнего хеша. Поля ключа, влияющие на equals/hashCode, должны быть стабильны; равенство только по ID с хешем по прогрессу также нарушает контракт равных объектов.

Задание 11 Выбор по нагрузке

Для шести сценариев — каталог по ID, отсортированный leaderboard, очередь, уникальные теги с порядком, диапазонный поиск, частый доступ по индексу — выберите интерфейс и реализацию. Укажите сложность главных операций и цену памяти/порядка.

Для каждого сценария выбран интерфейс и реализация. В рейтинге один балл связан со списком участников, чтобы равные баллы не затирали друг друга. Оценки для хеш-коллекций предполагают хорошее распределение хешей.

Сценарий	Интерфейс и реализация	Операции	Память и порядок
Каталог по ID	Map / HashMap	get, put: ожидаемо O(1)	O(n); корзины и узлы, без гарантии порядка
Рейтинг	NavigableMap / TreeMap; балл → List	Поиск группы: O(log n); добавление в ArrayList амортизированно O(1)	O(n + p); дерево и списки; баллы по убыванию
Очередь	Queue / ArrayDeque	offer, poll, peek: амортизированно O(1)	O(n); массив с запасом, FIFO
Теги с порядком	Set / LinkedHashSet	add, contains: ожидаемо O(1)	O(n); таблица и связи порядка
Диапазон ID	NavigableMap / TreeMap	get, put: O(log n); поиск и обход k результатов: O(log n + k)	O(n); узлы дерева, порядок по ID
Доступ по индексу	List / ArrayList	get: O(1); add в конец амортизированно O(1); remove из середины O(n)	O(n); массив с запасом, порядок позиций

В рейтинге n - число различных баллов, p - число участников; в остальных строках n - число элементов. Коллизии замедляют хеш-операции. subMap возвращает представление диапазона; оценка включает обход найденных записей.

Задание 12 Обобщенный репозиторий

Спроектируйте Repository<ID,T> с save, findById, findAll, deleteById. In-memory реализация не отдаёт наружу изменяемую внутреннюю коллекцию, не принимает null, определяет семантику повторного save.

Repository<ID,T> содержит save(ID,T), findById, findAll и deleteById. InMemoryRepository использует LinkedHashMap. Повторный save заменяет сущность того же ID без добавления второй записи. findById возвращает Optional, deleteById - факт удаления; null для ID и сущности запрещен.

findAll возвращает неизменяемый снимок списка. После обновления курса старый снимок сохраняет прежнюю запись; clear вызывает UnsupportedOperationException. Сами сущности не клонируются. Поиск, сохранение и удаление ожидаемо O(1), получение снимка - O(n).

Задание 13 Обратный индекс

По курсам и тегам постройте Map<Tag, Set<CourseId>>; поддержите добавление, удаление курса и пересечение нескольких тегов. Пустые set не остаются в индексе. Обоснуйте реализацию каждой коллекции.

TagIndex хранит Map<Tag,Set<CourseId>> через LinkedHashMap и LinkedHashSet: сохраняются порядок, уникальность ID и ожидаемо быстрый поиск. Добавление уже известного курса заменяет его теги. При удалении курса обходятся наборы ID; пустые наборы удаляются из map. Дополнительная обратная карта не используется.

Пересечение выполняется через retainAll: запрос java и backend дает курс 101. После удаления курса 101 тег backend исчезает из индекса. Неизвестный тег и пустой запрос дают пустое множество. Snapshot возвращает копии с запретом изменения как map, так и вложенных set.

Задание 14 Ограниченный LRU кэш

Реализуйте generic LruCache<K,V> фиксированной ёмкости на стандартных коллекциях. get обновляет недавность, превышение удаляет старейший элемент, null-политика явна. Докажите сценариями порядок вытеснения и стабильность ключей.

LruCache<K,V> использует LinkedHashMap с accessOrder=true. Успешный get и put делают запись самой недавней, промах не меняет порядок. При превышении положительной ёмкости удаляется первый, то есть наименее недавно использованный ключ. Null-ключи и значения запрещены, get возвращает Optional.

При ёмкости 2: put(A), put(B) дают [A, B]; get(A) - [B, A]; put(C) - [A, C], вытеснен B. Обновление A дает [C, A], затем put(D) - [A, D], вытеснен C. CourseId неизменяем: изменение прогресса значения Course не мешает найти запись по равному ID. get/put ожидаемо O(1), память O(ёмкости).

Задание 15 Планировщик prerequisites

Курсы образуют ориентированный граф зависимостей. Постройте generic-модель графа на map/set, найдите допустимый порядок изучения или цикл, выдайте диагностический путь цикла. Результат детерминирован при одинаковом вводе. Защитите выбор коллекций, сложности и поведение при отсутствующей вершине.

DependencyGraph<T> хранит ребра course → prerequisite в Map<T,Set<T>>. LinkedHashMap и LinkedHashSet фиксируют порядок вставки и исключают повторные ребра. DependencyPlanner выполняет DFS: сначала prerequisites, затем зависимый курс. Поэтому одинаковый ввод дает одинаковый порядок.

Состояния обхода: вершина в текущем пути или полностью обработана. Возврат к вершине текущего пути выявляет цикл; PlanningResult содержит либо полный порядок, либо замкнутый путь цикла без частичного порядка. Обе вершины ребра добавляются заранее. Неизвестная вершина вызывает IllegalArgumentException до изменения графа; пустой граф дает пустой допустимый порядок.

Для цепочки получен порядок [101, 102, 103, 104]. После добавления зависимости 101 от 103 обнаружен цикл [101, 103, 102, 101]; повторные запуски совпадают. Время O(V + E), память со снимком графа O(V + E), глубина стека DFS O(V). Ключи вершин должны иметь стабильные equals/hashCode.